Branch-Predictor Design Exploration on an Out-of-Order RISC-V Processor

Wonju Lee Shivam Kak

Abstract

We add a configurable front-end branch predictor to the lab's single-issue out-of-order RISC-V processor (`riscvooo`, the Lab 4 deliverable with reorder buffer and scoreboard) and evaluate three predictor algorithms of increasing complexity: a 1-bit BHT, a 2-bit saturating-counter BHT, and a two-level adaptive predictor that XORs the program counter with an 8-bit global history register to index a single shared pattern history table (the GShare variant from McFarling 1993). On top of these, we implement two stretch goals from the proposal — an 8-deep return-address stack (RAS) and F-stage JAL prediction — gated behind a single `BP_ENABLED` define so the unmodified path is byte-equivalent to lab4. Across the four lab 4 ubmarks, the 2-bit BHT lifts mean IPC from a baseline of 0.741 to 0.786 (+6.1%), and adding JAL prediction takes that to 0.799 (+7.8%), which is the winning configuration. The two-level predictor consistently underperforms BHT-2 on this workload because the kernels are dominated by single-direction loop branches that pollute the GHR. The RAS is functionally correct but is exercised only once or twice per kernel. Baseline `*-ooo.out` files are byte-identical to the lab4 reference outputs.

1 Design

1.1 Overview and proposal scope

The original proposal targeted "the dual-issue in-order superscalar RISC-V processor we have designed for Lab 3," but the actual lab tree contains a single-issue in-order seven-stage pipeline (`riscvlong`) and a single-issue out-of-order pipeline with reorder buffer (`riscvooo`, the Lab 4 deliverable). This project targets `riscvooo` so the predictor is integrated with the actual term-of-record OoO core, which means the misprediction penalty is two cycles (squash F + D when a conditional branch is wrong-path) rather than the three cycles the proposal cited for an X0-resolution dual-issue design. All three predictor algorithms apply unchanged. The proposal also called for an FPGA synthesis flow on a Nexys-4 DDR / Artix-7 part, which is deferred because the lab bench host became unreachable across the project window (sshd accepted TCP on port 22 but stopped sending its banner, and without bench access no one was able to power-cycle it). The writeup below covers IPC only.

1.2 Predictor sub-package

All seven predictor source files live in a new `bp/` sub-package that follows lab4's mcppbs convention. The predecoder (`bp_predecode.v`) is a pure-combinational decoder over the F-stage instruction word that classifies the opcode as a B-type branch, JAL, or JALR; recognises calls and returns ($\texttt{is\_call} = (\texttt{is\_jal} \vee \texttt{is\_jalr}) \wedge \texttt{rd}{=}x_1$ and $\texttt{is\_return} = \texttt{is\_jalr} \wedge \texttt{rs1}{=}x_1 \wedge \texttt{rd}{\neq}x_1$); and computes both the branch and JAL targets from F-stage bits alone: $\texttt{br\_target} = \texttt{pc} + \text{sext}(\texttt{imm\_sb})$ for B-type and $\texttt{jal\_target} = \texttt{pc} + \text{sext}(\texttt{imm\_uj})$ for JAL. JALR's target requires `rs1` and is left to the existing 1-cycle D-stage redirect.

The 1-bit BHT (`bp_bht1.v`) is a 256-entry register array with one bit per entry, indexed by `pc`[9:2] on read and on write. The 2-bit BHT (`bp_bht2.v`) replaces each entry with the standard `00/01/10/11` saturating-counter scheme; the prediction is the upper bit, and on reset all entries initialise to `01` (weakly not-taken).

The two-level predictor (`bp_two_level.v`) is a two-level adaptive scheme in the Yeh & Patt 1991 family. Level-1 is an 8-bit global branch history register (GHR) shared across all branches. Level-2 is a single 256-entry pattern history table (PHT) of 2-bit saturating counters. The PHT is indexed by $\texttt{pc}[9{:}2] \oplus \texttt{ghr}[7{:}0]$, the specific PC-XOR-history variant that McFarling 1993 named "GShare," and the GHR shifts in the resolved direction of every committed conditional branch. The GHR is only updated at *resolve* time (X), not at predict time (F). This avoids needing a checkpoint stack to roll back the GHR on a mispredict. The cost is that branches in the F→D→X shadow of an as-yet-unresolved branch see a slightly stale GHR, which is acceptable for the IPC range these benchmarks reach.

The return address stack (`bp_ras.v`) is an 8-entry × 32-bit LIFO. On a call, F pushes $\texttt{pc} + 4$; on a return, F pops the top and uses it as the predicted JALR target. The selectable wrapper (`bp_top.v`) instantiates exactly one

of the four direction predictors based on the BP_* define set at build time (BP_STATIC_NT, BP_BHT1, BP_BHT2, or BP_TWO_LEVEL). The pre-decoder, predictor instantiation, and pipeline registers are all gated by ‘ifdef BP_ENABLED, so the unmodified path through riscvooo is bit-equivalent to lab4 when BP_DEFINES is empty.

1.3 Integration into riscvooo

The integration patches affect four files: riscvooo-Core.v (signal wiring), riscvooo-CoreCtrl.v (predictor instantiation, F-stage redirect, X-stage resolve, diagnostic counters), riscvooo-CoreDpath.v (PC-mux extension and pred_target pipeline register), and riscvooo-sim.v (printing the new diagnostic counters in the output *-ooo.out file). Every change is gated by ‘ifdef BP_ENABLED, so the OoO machinery downstream of F→D — the scoreboard and the reorder buffer we implemented in Lab 4 — is untouched. Wrong-path instructions are squashed in F+D *before* they enter issue, which means the ROB never sees an instruction that won't commit; no checkpoint stack, no rollback machinery, and no ROB-flush logic are required. The predictor is purely a front-end addition.

The pre-decoder runs combinationally on the imemresp_queue mux output — the 32-bit instruction word that D will receive next cycle — tagged with pc_Fhl, and the predictor is queried with the same PC. The F stage fires a redirect through a new PC-mux input when the instruction is a predicted-taken B-type, when it is a JAL (under BP_PRED_JAL), or when it is a JALR-return whose RAS top is valid (under BP_RAS). Target priority is RAS, then JAL, then B-type. The PC-mux ordering becomes (highest priority first) brj_taken_Xhl → brj_taken_Dhl → pred_redirect_Fhl → pc + 4, with a new mux input pm_pred = 3 feeding pred_target_Fhl into the PC register on the next clock edge; the pc_mux_sel_Phl control signal grows from two bits to three bits when the predictor is enabled. A small register chain pipelines $\texttt{predict\_taken}_F$ along with the is_jal and ras_predict flags from F→D→X so the X-stage compare can determine mispredicts; the bit is valid only when the X-stage instruction is itself a B-type (br_sel_Xhl ≠ br_none), and the X-stage compare gates on bp_resolve_Xhl so stray "predictions" on non-branches never pollute the predictor's state.

The most important change in the pipeline is brj_taken_Xhl = mispredict_Xhl. With the predictor on, the X-stage redirect only fires when the prediction was wrong: a correct taken-prediction does not squash F+D because F is already fetching from the correct target, and a correct not-taken-prediction also does not squash. Correct predictions cost zero cycles, and that is where the IPC improvement comes from. On a mispredict the redirect target switches between branch_targ (actual taken) and pc_Xhl + 4 (actual not-taken) via a new redirect_to_target_Xhl signal back to the dpath. Without prediction, JAL and JALR raise brj_taken_Dhl to redirect F in D (the existing 1-cycle penalty); under JAL prediction the redirect is suppressed when is_jal_Dhl matches the pipelined pred_jal_Dhl flag, and on a JALR-return whose RAS prediction matches the actual register-computed target a 32-bit equality check in dpath (pred_target_match_Dhl) gates the D-stage redirect off. Wrong RAS predictions still cost the existing 1-cycle penalty.

2 Methodology

We deliberately constrained the design so that the build flow, toolchain, and output format match lab4's exactly. With BP_DEFINES unset, the project produces *-ooo.out files that are byte-identical to lab4's reference outputs for all 47 asm tests and all four ubmarks; only the trailing $finish line-number annotation differs, because riscvooo-sim.v grew seven lines for the ‘ifdef BP_ENABLED diagnostic prints. Predictor variants are selected by passing one or more BP_* defines at make time:

```
make BP_DEFINES="-DBP_ENABLED -DBP_BHT2 -DBP_PRED_JAL -DBP_RAS" \
     riscvooo-sim check-asm-riscvooo run-bmark-riscvooo
```

The cycle and instruction counts are read from proc.ctrl.num_cycles and proc.ctrl.num_inst — the same registers riscvooo-sim.v reports in lab4 — gated by stats_en ∨ csr_stats. The ubmark make target passes +verbose=1 (no +stats), so the counters fire only inside the test_stats_on / test_stats_off kernel region of each benchmark, which is the convention lab4 uses; the asm-test rule passes +stats=1 and counts the whole program. Five new BP_ENABLED-gated counters — num_branches, num_taken, num_pred_resolve, num_pred_correct, num_mispredicts — share the same gating clause.

We sweep nine BP variants: baseline, static_nt, BHT-1, BHT-2, two-level, plus three pairings of BHT-2 and the two-level predictor with JAL and JAL+RAS. Each variant is a clean rebuild (∼10 s) followed by all 47 asm tests and 4 ubmarks (∼5 s), and the full sweep takes about 90 seconds on Princeton's adroit-vis cluster. All

47 asm-test targets in `l4/lab4/build/Makefile` pass on all nine variants ($47 \times 9 = 423$ runs, all `[ PASSED ]`). This includes the standard branch-and-jump suite (`riscv-beq`, `riscv-bne`, `riscv-blt`, `riscv-bge`, `riscv-bltu`, `riscv-bgeu`, `riscv-j`, `riscv-jal`, `riscv-jalr`, `riscv-jr`), the integer/load-store/mul-div suite, and the four OoO-specific tests we wrote in Lab 4 (`riscv-ooo-commit`, `riscv-rob-bypass`, `riscv-waw`, `riscv-long-better-ipc`). All four ubmarks (`vvadd`, `cmplx-mult`, `bin-search`, `masked-filter`) report `*** PASSED ***` on every variant. The predictor integration preserves architectural correctness in every configuration we tested.

3 Evaluation

Tables 1 and 2 report the kernel-only IPC for the four ubmarks across all nine variants, and Table 3 reports the per-variant mispredict counts on the N kernel-region conditional branches.

Table 1: IPC by variant (kernel-only, lab4 convention). Five-variant view; `static_nt` is the integration-overhead control.

Benchmark	baseline	static-NT	BHT-1	BHT-2	two-level (PC⊕GHR)
`vvadd`	0.8865	0.8865	0.9115	0.9115	0.8830
`cmplx-mult`	0.7105	0.7105	0.7236	0.7236	0.7194
`bin-search`	0.7048	0.7048	0.7865	0.7952	0.7664
`masked-filter`	0.6622	0.6622	0.7064	0.7153	0.7115
mean	**0.741**	**0.741**	**0.782**	**0.786**	**0.770**

Table 2: IPC with the proposal stretch goals (JAL prediction at F, 8-deep RAS for JALR returns) layered on top of BHT-2 and the two-level predictor. "full" = direction predictor + `BP_PRED_JAL` + `BP_RAS`.

Benchmark	BHT-2 + JAL	two-level + JAL	BHT-2 full	two-level full
`vvadd`	0.9115	0.8830	0.9115	0.8830
`cmplx-mult`	0.7239	0.7197	0.7239	0.7197
`bin-search`	0.8215	0.7908	0.8215	0.7908
`masked-filter`	0.7394	0.7354	0.7394	0.7354
mean	**0.799**	**0.782**	**0.799**	**0.782**

Table 3: Conditional branches in the kernel region and mispredicts per direction predictor. Percentages are mispredicts/branches.

Benchmark	branches	BHT-1	BHT-2	two-level (PC⊕GHR)
`vvadd`	10	2 (20.0%)	2 (20.0%)	10 (100.0%)
`cmplx-mult`	27	3 (11.1%)	3 (11.1%)	10 (37.0%)
`bin-search`	246	59 (24.0%)	52 (21.1%)	76 (30.9%)
`masked-filter`	668	96 (14.4%)	53 (7.9%)	71 (10.6%)

As a sanity check against lab4, the baseline `*-ooo.out` files diff cleanly against `l4/lab4/build/*-ooo.out`: 511/453/0.886 on `vvadd`, 2425/1723/0.711 on `cmplx-mult`, 1443/1017/0.705 on `bin-search`, and 7446/4931/0.662 on `masked-filter` (`cycles/inst/ipc`). The build flow and RTL behave bit-for-bit like lab4's reference for the unmodified core.

3.1 Discussion

The first thing to note is that `baseline` and `static_nt` produce identical IPC to four decimal places on every benchmark. The lab core's existing behaviour is itself "always not-taken" — fetch PC+4, correct on X resolution — so adding the predictor in `static_nt` mode should not change observable behaviour, and it does not. This validates

that the new pipeline registers, the widened PC mux, and the `‘ifdef BP_ENABLED` hooks introduce no correctness regression or measurable overhead when the predictor signal is constant. Among the actually-predicting configurations, BHT-1 and BHT-2 tie on three of four benchmarks; the exception is `masked-filter`, where BHT-2 cuts mispredicts almost in half ($96 \rightarrow 53$, -45%). That benchmark contains an inner branch whose direction alternates at a frequency that exposes BHT-1's weakness: a single anomalous outcome flips the prediction, so a nearly-deterministic mostly-taken branch with one rare not-taken iteration costs four mispredicts (one to flip out, one for the rare case, one to flip back, one for the next normal case). The 2-bit hysteresis absorbs the rare case without flipping the prediction.

The two-level predictor underperforms BHT-2 on every benchmark, and the most striking case is `vvadd`, where every single one of the ten kernel-region conditional branches is mispredicted. The underlying reason is GHR pollution: the kernel's branches are nearly all single-direction loop branches with no useful global-history correlation, but the GHR cycles through patterns of mostly-1s anyway, so the same PC maps to different PHT counters depending on how many other taken branches resolved recently. Each mapped counter then has only a handful of training cycles to bias toward "taken" before it gets remapped, so the warm-up cost dominates the kernel. BHT-2's per-PC counters do not suffer this — they accumulate hysteresis across all visits to the same branch. The general lesson is that the global-history correlation that the two-level scheme is built around only pays off when there is correlation to exploit; on `ubmark-vvadd/cmplx-mult/bin-search/ masked-filter`, there is none.

The cycle accounting on `vvadd` makes the small numbers in the table concrete. With `-O3 -funroll-loops` (matching lab4's `ubmark.mk`), the 100-iteration loop collapses into roughly 12 unrolled chunks; only ten conditional branches survive in the measured region, of which nine are taken. The baseline kernel runs in 511 cycles and retires 453 instructions (IPC 0.886). The static cost of mispredicting nine taken branches is $9 \times 2 = 18$ cycles. BHT-2 cuts that to two mispredicts $\times$ two cycles $= 4$ cycles, so the kernel completes in 497 cycles (IPC 0.911): a saving of 14 cycles, $\approx 78\%$ of the 18-cycle ceiling. The remaining gap is from cold predictor entries on the very first loop iteration plus a couple of mispredicts in the chunk-out tail where the unroll factor doesn't divide the iteration count evenly.

JAL prediction and the RAS together lift mean IPC from BHT-2's 0.786 to 0.799, but the contribution is uneven across benchmarks and across the two stretch goals. JAL prediction is the larger contributor: on `bin-search` it lifts IPC from 0.795 to 0.822 (+3.4%), and on `masked-filter` from 0.715 to 0.739 (+3.4%). `bin-search` contains the inner-loop call sites where this stretch goal pays off, while on `vvadd` and `cmplx-mult` the kernel JAL count is zero or one and the IPC delta is below counter resolution. The RAS, by contrast, is functionally correct but adds zero IPC on these benchmarks — `bp_bht2_full` produces the same numbers as `bp_bht2_jal` on every benchmark. The reason is that the `test_stats_on/off` kernel regions of these ubmarks contain at most a couple of JALR returns (most return-from-main happens *outside* `test_stats_off`), so on a workload with deeper recursion or more function calls inside the measured region the RAS would matter more. We verified RAS correctness in waveform: the `push_addr` on a JAL-with-rd=x_1 matches the next sequential PC, the `pop` on a JALR-with-rs1=x_1/rd=$\neq x_1$ recovers that PC, and `pred_target_match_Dhl` suppresses the D-stage redirect.

Putting these observations together, BHT-2 + JAL is the highest-IPC configuration across all nine design points at 0.799 mean, and BHT-1 is the smallest predictor that gets within 5% of it (0.782 mean). The two-level predictor costs more state than BHT-2 (an extra 8-bit GHR plus an XOR network on the index path) and produces strictly worse IPC on these workloads, so for this lab's benchmark set additional history correlation does not pay off; a workload with strongly correlated inter-branch behaviour would be expected to flip that conclusion. On a separate axis, we re-ran the same predictor variants on `riscvlong` (single-issue in-order, the Lab 2 deliverable) for comparison and observed two patterns: baseline IPC is consistently lower on `riscvooo` than on `riscvlong` for these workloads, in line with our Lab 4 report's conclusion that tight dependency chains expose the OoO front-end's extra pipeline state and in-order commit constraint without recovering the cost; and the *relative* IPC lift from the predictor is similar across the two cores, because the misprediction penalty is two cycles in both pipelines and the cycles the predictor saves are the same cycles in both.

4 Related work

The 1-bit and 2-bit BHT designs follow Smith [1], who introduced the saturating-counter scheme that all subsequent direction predictors build on. The two-level adaptive structure — a global history register feeding a pattern history table — is the Yeh & Patt [2, 3] family, and the specific PC$\oplus$GHR indexing variant we implemented is from McFarling [4], who also introduced tournament predictors that combine local and global predictors and showed GShare's accuracy benefit on SPEC at small storage budgets. The 8-deep return-address stack follows Kaeli & Emma [5], who first proposed using a stack to predict return-from-call targets in pipelined processors, and Hennessy & Patterson [8] surveys the trade-offs at textbook level. More recent academic work, particularly perceptron predictors [6] and

TAGE [7], substantially outperforms the predictors in this study on full-system workloads, but those designs require considerably more state and several pipelined stages of lookup, and they would not fit cleanly into the existing single-cycle F-stage of `riscvooo` without re-pipelining. The proposal originally called for an FPGA area/Fmax tradeoff in the spirit of McFarling and several follow-on FPGA branch-predictor studies; that analysis is the obvious next step for a future revision of this work once a synthesis bench is available.

5 Conclusion

This project added a configurable front-end branch predictor to the Lab 4 OoO core, evaluated three direction-predictor algorithms plus two stretch goals (JAL prediction and a return-address stack), and ran the four lab 4 ubmarks through all nine variants. The `BHT-2 + JAL` configuration is the winning point at 0.799 mean IPC (+7.8% over baseline). The two-level predictor with PC⊕GHR-indexed PHT consistently underperforms the simpler 2-bit BHT on this workload because the kernels contain many single-direction loop branches that pollute the GHR. The 8-deep RAS is functionally correct but the kernels exercise it only minimally; its mechanism is in place for workloads that contain deeper recursion or more function calls. The implementation lives in a new `bp/` sub-package and four lightly patched files in `riscvooo/`, the build flow uses lab4's existing autoconf pipeline plus one new `BP_DEFINES` knob, and the baseline `*-ooo.out` files are byte-identical to the lab4 reference. The full sweep is reproducible from a clean checkout with `source scripts/adroit-env.sh && ./scripts/run_variants.sh`, and the source is at `https://github.com/Shivamkak19/fpga_riscv_branch_predictor` on the `main` branch.

References

[1] J. E. Smith. *A study of branch prediction strategies.* ISCA, 1981.

[2] T.-Y. Yeh and Y. N. Patt. *Two-level adaptive training branch prediction.* MICRO, 1991.

[3] T.-Y. Yeh and Y. N. Patt. *Alternative implementations of two-level adaptive branch prediction.* ISCA, 1992.

[4] S. McFarling. *Combining branch predictors.* WRL Technical Note TN-36, 1993.

[5] D. R. Kaeli and P. G. Emma. *Branch history table prediction of moving target branches due to subroutine returns.* ISCA, 1991.

[6] D. A. Jiménez and C. Lin. *Dynamic branch prediction with perceptrons.* HPCA, 2001.

[7] A. Seznec and P. Michaud. *A case for (partially) tagged geometric history length branch prediction.* JILP, 2006.

[8] J. L. Hennessy and D. A. Patterson. *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017. Chapter on branch prediction.